

composer/xdebug-handler



Restart a CLI process without loading the Xdebug extension, unless `xdebug.mode=off`.

Originally written as part of `composer/composer`, now extracted and made available as a stand-alone library.

Version 3

Removed support for legacy PHP versions and added type declarations.

Long term support for version 2 (PHP 5.3.2 - 7.2.4) follows Composer 2.2 LTS policy.

Installation

Install the latest version with:

```
$ composer require composer/xdebug-handler
```

Requirements

- PHP 7.2.5 minimum, although using the latest PHP version is highly recommended.

Basic Usage

```
use Composer\XdebugHandler\XdebugHandler;

$xdebug = new XdebugHandler('myapp');
$xdebug->check();
unset($xdebug);
```

The constructor takes a single parameter, `$envPrefix`, which is upper-cased and prepended to default base values to create two distinct environment variables. The above example enables the use of:

- `MYAPP_ALLOW_XDEBUG=1` to override automatic restart and allow Xdebug
- `MYAPP_ORIGINAL_INIS` to obtain ini file locations in a restarted process

Advanced Usage

- How it works
- Limitations
- Helper methods
- Setter methods

- Process configuration
- Troubleshooting
- Extending the library
- Examples

How it works

A temporary ini file is created from the loaded (and scanned) ini files, with any references to the Xdebug extension commented out. Current ini settings are merged, so that most ini settings made on the command-line or by the application are included (see Limitations)

- MYAPP_ALLOW_XDEBUG is set with internal data to flag and use in the restart.
- The command-line and environment are configured for the restart.
- The application is restarted in a new process.
 - The restart settings are stored in the environment.
 - MYAPP_ALLOW_XDEBUG is unset.
 - The application runs and exits.
- The main process exits with the exit code from the restarted process.

See Examples for further information.

Signal handling Asynchronous signal handling is automatically enabled if the pcntl extension is loaded. SIGINT is set to SIG_IGN in the parent process and restored to SIG_DFL in the restarted process (if no other handler has been set).

From PHP 7.4 on Windows, CTRL+C and CTRL+BREAK handling is automatically enabled in the restarted process and ignored in the parent process.

Limitations

There are a few things to be aware of when running inside a restarted process.

- Extensions set on the command-line will not be loaded.
- Ini file locations will be reported as per the restart - see `getAllIniFiles()`.
- Php sub-processes may be loaded with Xdebug enabled - see Process configuration.

Helper methods

These static methods provide information from the current process, regardless of whether it has been restarted or not.

getAllIniFiles(): array Returns an array of the original ini file locations. Use this instead of calling `php_ini_loaded_file` and `php_ini_scanned_files`, which will report the wrong values in a restarted process.

use `Composer\XdebugHandler\XdebugHandler;`

```
$files = XdebugHandler::getAllIniFiles();
```

```
# $files[0] always exists, it could be an empty string
$loadedIni = array_shift($files);
$scannedInis = $files;
```

These locations are also available in the MYAPP_ORIGINAL_INIS environment variable. This is a path-separated string comprising the location returned from `php_ini_loaded_file`, which could be empty, followed by locations parsed from calling `php_ini_scanned_files`.

getRestartSettings(): ?array Returns an array of settings that can be used with PHP sub-processes, or null if the process was not restarted.

```
use Composer\XdebugHandler\XdebugHandler;
```

```
$settings = XdebugHandler::getRestartSettings();
/**
 * $settings: array (if the current process was restarted,
 * or called with the settings from a previous restart), or null
 *
 * 'tmpIni'      => the temporary ini file used in the restart (string)
 * 'scannedInis' => if there were any scanned inis (bool)
 * 'scanDir'     => the original PHP_INI_SCAN_DIR value (false/string)
 * 'phprc'       => the original PHPRC value (false/string)
 * 'inis'        => the original inis from getAllIniFiles (array)
 * 'skipped'     => the skipped version from getSkippedVersion (string)
 */
```

getSkippedVersion(): string Returns the Xdebug version string that was skipped by the restart, or an empty string if there was no restart (or Xdebug is still loaded, perhaps by an extending class restarting for a reason other than removing Xdebug).

```
use Composer\XdebugHandler\XdebugHandler;
```

```
$version = XdebugHandler::getSkippedVersion();
# $version: '3.1.1' (for example), or an empty string
```

isXdebugActive(): bool Returns true if Xdebug is loaded and is running in an active mode (if it supports modes). Returns false if Xdebug is not loaded, or it is running with `xdebug.mode=off`.

Setter methods

These methods implement a fluent interface and must be called before the main `check()` method.

setLogger(LoggerInterface \$logger): self Enables the output of status messages to an external PSR3 logger. All messages are reported with either DEBUG or WARNING log levels. For example (showing the level and message):

```
// No restart
DEBUG    Checking MYAPP_ALLOW_XDEBUG
DEBUG    The Xdebug extension is loaded (3.1.1) xdebug.mode=off
DEBUG    No restart (APP_ALLOW_XDEBUG=0) Allowed by xdebug.mode

// Restart overridden
DEBUG    Checking MYAPP_ALLOW_XDEBUG
DEBUG    The Xdebug extension is loaded (3.1.1) xdebug.mode=coverage,debug,develop
DEBUG    No restart (MYAPP_ALLOW_XDEBUG=1)

// Failed restart
DEBUG    Checking MYAPP_ALLOW_XDEBUG
DEBUG    The Xdebug extension is loaded (3.1.0)
WARNING  No restart (Unable to create temp ini file at: ...)
```

Status messages can also be output with XDEBUG_HANDLER_DEBUG. See Troubleshooting.

setMainScript(string \$script): self Sets the location of the main script to run in the restart. This is only needed in more esoteric use-cases, or if the `argv[0]` location is inaccessible. The script name `--` is supported for standard input.

setPersistent(): self Configures the restart using persistent settings, so that Xdebug is not loaded in any sub-process.

Use this method if your application invokes one or more PHP sub-process and the Xdebug extension is not needed. This avoids the overhead of implementing specific sub-process strategies.

Alternatively, this method can be used to set up a default *Xdebug-free* environment which can be changed if a sub-process requires Xdebug, then restored afterwards:

```
function SubProcessWithXdebug()
{
    $phpConfig = new Composer\XdebugHandler\PhpConfig();

    # Set the environment to the original configuration
    $phpConfig->useOriginal();

    # run the process with Xdebug loaded
    ...
}
```

```

    # Restore Xdebug-free environment
    $phpConfig->usePersistent();
}

```

Process configuration

The library offers two strategies to invoke a new PHP process without loading Xdebug, using either *standard* or *persistent* settings. Note that this is only important if the application calls a PHP sub-process.

Standard settings Uses command-line options to remove Xdebug from the new process only.

- The `-n` option is added to the command-line. This tells PHP not to scan for additional inis.
- The temporary ini is added to the command-line with the `-c` option.

If the new process calls a PHP sub-process, Xdebug will be loaded in that sub-process (unless it implements xdebug-handler, in which case there will be another restart).

This is the default strategy used in the restart.

Persistent settings Uses environment variables to remove Xdebug from the new process and persist these settings to any sub-process.

- `PHP_INI_SCAN_DIR` is set to an empty string. This tells PHP not to scan for additional inis.
- `PHPRC` is set to the temporary ini.

If the new process calls a PHP sub-process, Xdebug will not be loaded in that sub-process.

This strategy can be used in the restart by calling `setPersistent()`.

Sub-processes The `PhpConfig` helper class makes it easy to invoke a PHP sub-process (with or without Xdebug loaded), regardless of whether there has been a restart.

Each of its methods returns an array of PHP options (to add to the command-line) and sets up the environment for the required strategy. The `getRestartSettings()` method is used internally.

- `useOriginal()` - Xdebug will be loaded in the new process.
- `useStandard()` - Xdebug will **not** be loaded in the new process - see standard settings.
- `userPersistent()` - Xdebug will **not** be loaded in the new process - see persistent settings

If there was no restart, an empty options array is returned and the environment is not changed.

```
use Composer\XdebugHandler\PhpConfig;

$config = new PhpConfig;

$options = $config->useOriginal();
# $options:      empty array
# environment:   PHPRC and PHP_INI_SCAN_DIR set to original values

$options = $config->useStandard();
# $options:      [-n, -c, tmpIni]
# environment:   PHPRC and PHP_INI_SCAN_DIR set to original values

$options = $config->usePersistent();
# $options:      empty array
# environment:   PHPRC=tmpIni, PHP_INI_SCAN_DIR=''
```

Troubleshooting

The following environment settings can be used to troubleshoot unexpected behavior:

- `XDEBUG_HANDLER_DEBUG=1` Outputs status messages to `STDERR`, if it is defined, irrespective of any PSR3 logger. Each message is prefixed `xdebug-handler[pid]`, where `pid` is the process identifier.
- `XDEBUG_HANDLER_DEBUG=2` As above, but additionally saves the temporary ini file and reports its location in a status message.

Extending the library

The API is defined by classes and their accessible elements that are not annotated as `@internal`. The main class has two protected methods that can be overridden to provide additional functionality:

requiresRestart(bool \$default): bool By default the process will restart if Xdebug is loaded and not running with `xdebug.mode=off`. Extending this method allows an application to decide, by returning a boolean (or equivalent) value. It is only called if `MYAPP_ALLOW_XDEBUG` is empty, so it will not be called in the restarted process (where this variable contains internal data), or if the restart has been overridden.

Note that the `setMainScript()` and `setPersistent()` setters can be used here, if required.

restart(array \$command): void An application can extend this to modify the temporary ini file, its location given in the `tmpIni` property. New settings can be safely appended to the end of the data, which is `PHP_EOL` terminated.

The `$command` parameter is an array of unescaped command-line arguments that will be used for the new process.

Remember to finish with `parent::restart($command)`.

Example This example demonstrates two ways to extend basic functionality:

- To avoid the overhead of spinning up a new process, the restart is skipped if a simple help command is requested.
- The application needs write-access to phar files, so it will force a restart if `phar.readonly` is set (regardless of whether Xdebug is loaded) and change this value in the temporary ini file.

```
use Composer\XdebugHandler\XdebugHandler;
use MyApp\Command;

class MyRestarter extends XdebugHandler
{
    private $required;

    protected function requiresRestart(bool $default): bool
    {
        if (Command::isHelp()) {
            # No need to disable Xdebug for this
            return false;
        }

        $this->required = (bool) ini_get('phar.readonly');
        return $this->required || $default;
    }

    protected function restart(array $command): void
    {
        if ($this->required) {
            # Add required ini setting to tmpIni
            $content = file_get_contents($this->tmpIni);
            $content .= 'phar.readonly=0'.PHP_EOL;
            file_put_contents($this->tmpIni, $content);
        }

        parent::restart($command);
    }
}
```

Examples

The `tests\App` directory contains command-line scripts that demonstrate the internal workings in a variety of scenarios. See [Functional Test Scripts](#).

License

composer/xdebug-handler is licensed under the MIT License, see the [LICENSE](#) file for details.